ContractKV: Certified Prompt Compilation for Transactional Prefix Reuse in Multi-Tenant LLM Services

Bin Zhou and Xi Cao, *Member, IEEE*

***Abstract*—A retrieval service can repeat thousands of document tokens while changing a question placed before them. Native prefix caching then misses most of the repeated context. Reordering exposes a longer prefix, but can change model output and propagate unsupported state across tenants. We present ContractKV, which treats a compiled KV prefix as a service object admitted by scoped behavioral evidence and made visible through a transaction. Contract-Aware Layout Compilation preserves registered dependencies; Scoped Behavioral Certification checks a model-generated token path and termination; Prefix-Selective Transactional Promotion commits only the exact, block-aligned stable prefix; and Service-Template Certificates bind authorization and enable qualification costs to be amortized. We distinguish deployment-qualified templates from a stricter query-qualified protocol whose first raw business response supplies the witness. On six source-disjoint HotpotQA and RepoBench templates, three fresh-process repetitions yield full-cost throughput gains of 1.619×, 1.109×, and 1.070× for Qwen3-8B, Mistral-7B, and Llama-3.1-8B. All 1,296 admitted compiled future pairs retain their raw output token IDs. These zero-slack measurements include successful and failed qualification and waiting; p99 does not uniformly improve. A separate Qwen cohort yields 1.803×. Live rejection, template-amortization, and cache-lifecycle experiments connect the gains to controlled publication and physical reuse. The results show where behavior-scoped compilation can turn prompt repetition into a measurable service benefit.**

***Index Terms*—Large language model serving, KV cache, prefix caching, behavioral certification, transactional publication, multi-tenant services.**

I. Introduction

Consider a retrieval service that sends the current question before a long set of source documents. Two authorized users ask different questions about the same documents. Almost all of the document text repeats, yet native prefix caching stops at the first changed question token. This order occurs in a fixed revision of Azure's public RAG demonstration, whose conversation history and question precede the retrieved sources [1]. The opportunity is concrete: move reusable context ahead of changing fields, and subsequent requests can reuse a longer physical KV prefix.

The difficulty is deciding when that transformed state is suitable for a shared service. A dependency-valid reorder can still change an autoregressive model's answer. A passing check can be applied outside the inputs for which it supplied evidence. A speculative candidate can also enter the shared cache before its qualification finishes. These are linked service decisions: admissible layout, applicable behavioral evidence, and authorized publication. Addressing only the layout leaves later tenants dependent on an unchecked execution artifact.

Existing systems establish several ways to exploit repeated context. PagedAttention and RadixAttention manage exact-prefix sharing [2], [3]; Prompt Cache exposes reusable modules [4]; CacheBlend and EPIC reuse context across positions through different execution mechanisms [5], [6]. Application-aware systems such as Parrot expose dependencies to the serving layer [7]. ContractKV addresses the complementary control-plane question: *what evidence and identity permit a transformed prefix to become shared state?*

Our central idea is to make that prefix a versioned service object. The same binding that authorizes a compiled layout governs the state visible to later requests. Four mechanisms implement this idea. The *Contract-Aware Layout Compiler* (CALC) exposes stable leading content while preserving registered dependencies. *Scoped Behavioral Certification* associates model-backed token-path evidence with an input and execution scope. *Prefix-Selective Transactional Promotion* (PSTP) keeps candidate state private until a valid pass, then commits only the eligible stable blocks. A *Service-Template Certificate* binds identity, authorization, and lifecycle so that qualified requests can share the object and its admission cost.

Qualification granularity is part of the service contract. We evaluate two protocols within this architecture. Deployment-qualified execution registers an already qualified template route using a representative compiled trajectory. Query-qualified execution uses the first real raw business response for each query scope as its witness, then shares the decision across authorized requests in that scope. The latter provides the stricter free-running evaluation in this paper. It requires work proportional to distinct selected query scopes, rather than one behavioral decision for arbitrary future queries. The template prefix remains a common physical reuse object across those decisions.

The primary evaluation asks whether that stricter protocol still pays for itself on preserved task inputs. Across six source-disjoint HotpotQA and RepoBench templates, three models, and three fresh-process repetitions, ContractKV achieves 1.619×, 1.109×, and 1.070× full-cost throughput. The service clock includes raw witnesses, failed attempts, qualification, and waiting, with zero tool slack. All 1,296 admitted compiled future pairs match their raw output IDs. Coverage and absolute task scores remain visible, and p99 increases are reported.

B. Zhou is with Beijing University of Chemical Technology, Beijing 102218, China (e-mail: 2025200800@buct.edu.cn).

X. Cao is with Beijing University of Chemical Technology, Beijing 102218, China (e-mail: caoxi@buct.edu.cn).

Corresponding author: Xi Cao.

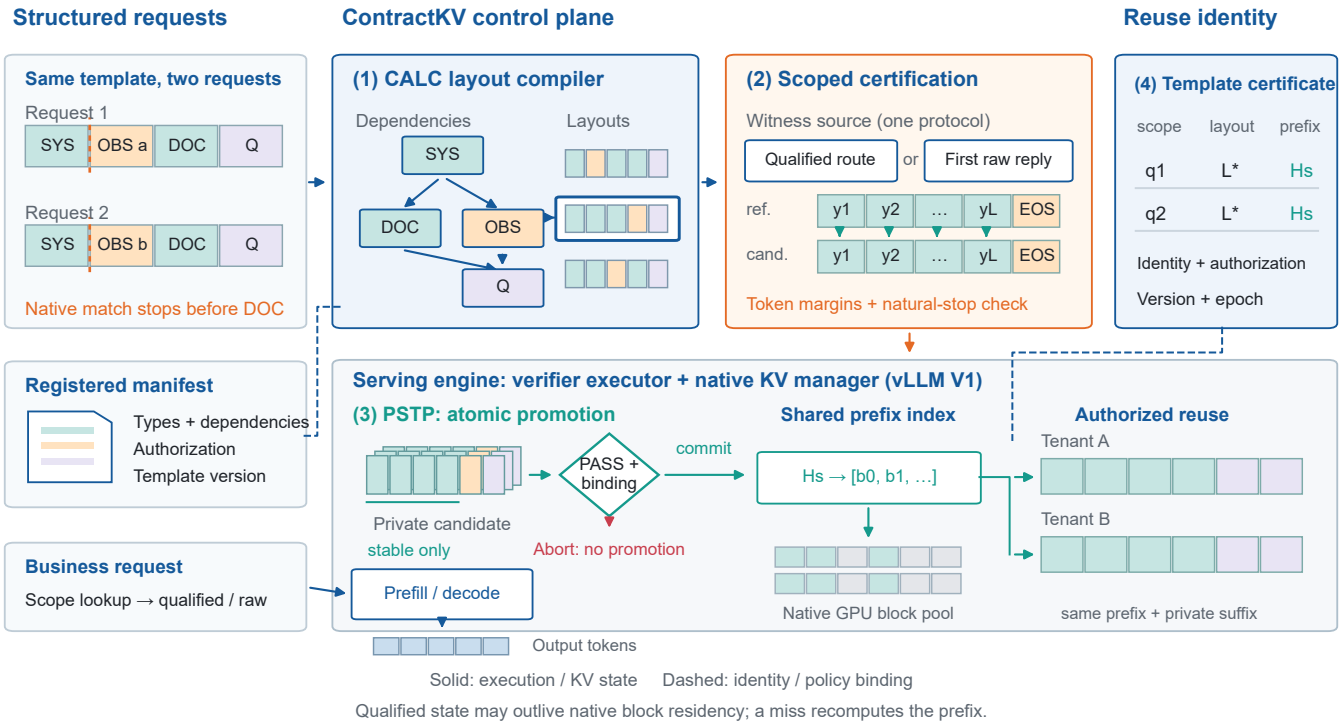


Fig. 1. ContractKV makes a compiled stable prefix an evidence-bound service object. CALC repairs a structural prefix gap; one of two separately evaluated witness protocols supplies scoped token-path evidence; PSTP commits eligible state only after PASS and a valid binding. Authorized tenants of the same template share the prefix and keep changing suffixes private. Qualification and physical residency have separate lifetimes.

Separate deployment experiments demonstrate qualification amortization and controlled publication; block-level probes distinguish valid qualification from the continued residency needed to realize a hit.

The paper makes four contributions, each connecting a mechanism to observable service behavior:

- **Contract-aware compilation** of bounded, dependency-valid layout candidates with exact rendered-token identities (Section IV-B).
- **Scoped model-backed admission** that binds token-path and termination evidence to declared inputs and execution conditions, evaluated with real free-running generation (Sections IV-C and VI-C).
- **Transactional KV publication** at an exact-LCP, block-aligned commit boundary, with zero promotion for rejected candidates (Sections IV-D and VI-E).
- **Service-template reuse** that separates behavioral decision granularity from physical prefix identity and measures amortization, full service cost, and cache availability (Sections IV-E, VI-B, and VI-E1).

II. Background and Motivation

A. The Gap Between Repetition and Prefix Reuse

A causal language model first computes prompt states during prefill, then generates output autoregressively. Native prefix caching reuses a leading sequence whose tokens and cache identity match a resident entry [2], [3]. For fully rendered prompt $\mathbf{t}$, let $\mathcal{P}(\mathcal{R})$ be the token prefixes represented by resident state under the matching cache identity. The block-aligned reuse opportunity is

$$\begin{aligned} k^* &= \max\{k : \mathbf{t}_{1:k} \in \mathcal{P}(\mathcal{R})\}, \\ k_B(\mathbf{t}, \mathcal{R}) &= B\,\lfloor k^*/B \rfloor, \end{aligned} \tag{1}$$

where B is the block size. Byte-identical text outside that leading sequence offers a layout opportunity. Its KV state also depends on preceding tokens and positions [2].

A concrete application example comes from a fixed revision of Azure's RAG demonstration: its conversation builder appends history before the current user message, and its user template places the query before the retrieved sources [1]. If those earlier fields change while some sources repeat, native prefix continuity ends before the repeated sources. This code example establishes a realizable request order; the measurement below independently quantifies the effect in controlled template traces.

B. Measuring the Structural Opportunity

We reconstruct stability and native reuse from actual API prompt IDs, the frozen renderer's segment spans, and scheduler-recorded prefix hits. Let n_r be the prompt length, S_r the set of token positions fully contained in manifest-declared stable body segments, and h_r the measured native hit length. Three distinct token-weighted quantities are

$$\begin{aligned} \rho_s &= \frac{\sum_r |S_r|}{\sum_r n_r}, \qquad \rho_h = \frac{\sum_r h_r}{\sum_r n_r}, \\ \rho_{s|h} &= \frac{\sum_r |S_r \cap \{1, \ldots, h_r\}|}{\sum_r |S_r|}. \end{aligned} \tag{2}$$

TABLE I
MEASURED STRUCTURAL OPPORTUNITY IN THE SIX-TEMPLATE, TASK-PRESERVING COHORT. EACH ROW CONTAINS 1,620 FUTURE RAW REQUESTS ACROSS THREE REPETITIONS. RATIOS ARE TOKEN-WEIGHTED AND USE EQ. (2).

Model	Stable body ρ_s	Native hits ρ_h	$\rho_{s\|h}$
Qwen3-8B	98.520%	0.950%	0.030%
Mistral-7B	98.477%	0.985%	0.034%
Llama-3.1-8B	98.357%	1.114%	0.031%

The first describes stable content, the second all native hits, and the third recovery of the stable body itself. The full-prompt denominator includes chat-control tokens and headings; the stable body excludes wrappers and cross-boundary tokens. Source-macro averages are calculated separately.

Table I shows that the task-preserving requests contain substantial repeated context but recover little of that stable body in raw order. The ratios describe controlled template traces, not production sharing frequencies. They are a new actual-ID reconstruction; the earlier table's exact numerical provenance was not recovered and its percentages are not reused.

C. From a Layout Opportunity to a Shared Service Object

Moving stable content forward addresses continuity, but leaves two decisions unresolved. First, the candidate must preserve the manifest's ordering constraints and obtain behavioral evidence for its intended use. Second, publishing candidate KV into a shared index must be all-or-nothing at an authorized boundary. A partially checked prefix, stale template identity, or overbroad qualification can affect requests other than the one that created it.

qualification authorizes reuse, publication exposes an eligible prefix, and residency determines the reuse actually obtained.

III. SERVICE MODEL AND DESIGN OBJECTIVES

A. Structured Requests and Registered Dependencies

A structured request $r = (s_1, \ldots, s_m)$ contains application segments. Each segment carries an identifier, original content, a stability type, and a registered content identity. We retain three types: STABLE, MUTABLE, and QUERY. Stable content belongs to an immutable template version; mutable content includes observations and changing state; query content carries the current task. A query may recur within an authorized directory while remaining outside the promoted stable prefix.

The operator manifest defines segment boundaries, separators, stability, dependencies, and the sharing domain. Unregistered tenant content remains private. A segment dependency graph $G_r = (V, E, \lambda)$ associates each edge with a data, causal, or positional ordering constraint. A layout π is legal when it is a topological order of G_r:

$$(s_i, s_j) \in E \quad \Longrightarrow \quad \mathrm{pos}_\pi(s_i) < \mathrm{pos}_\pi(s_j). \tag{3}$$

All cache identities and prefix boundaries are computed from the fully rendered prompt $\mathbf{t}_\pi$, including the model's chat template. Per-segment token counts guide construction, but exact boundaries follow retokenization; concatenation can change tokenization at segment edges. Let $P(\pi)$ denote the token boundary of the leading shareable stable content. CALC provides a bounded candidate set $\Pi_L(G_r)$ containing raw order L0 and registered compiled levels. Behavioral qualification selects among these legal candidates.

B. Templates, Qualifications, and Cache Identity

A service template

$$\Theta = (V, \mathbf{S}, \mathcal{A}, \mathcal{P}, \mathcal{M}, F) \tag{4}$$

binds a version V, stable content $\mathbf{S}$, an authorized tenant set $\mathcal{A}$, a frozen policy $\mathcal{P}$, its manifest $\mathcal{M}$, and declared fan-out F. The input may interleave stable and private segments; stable-first order is a compilation result.

ContractKV uses two related identities. A *behavioral qualification* determines whether a request may use a particular compiled layout. A *stable-prefix identity* identifies the corresponding reusable KV prefix within its authorized template namespace. Several admitted query scopes can map to the same stable-prefix identity.

Execution identity binds the model, tokenizer/chat template, decoding/stopping settings, policy and runtime. Section IV-E defines the complete template and lifecycle binding.

C. Publication and Service Objectives

ContractKV aims to turn admissible structural reuse into lower complete service cost while preserving the following publication properties.

P1: Evidence-gated publication. Candidate compiled state becomes shared only after a completed verifier pass and a valid publication binding.

P2: Stable-prefix confinement. PSTP publishes complete blocks within the stable, certified, exact-match boundary. Mutable and query suffixes are excluded from this promotion.

P3: Atomic failure handling. A failed, incomplete, expired, or conflicting transaction contributes no newly promoted candidate blocks. Raw-order execution remains the service fallback.

P4: Authorization-bound reuse. The deployment control path validates template identity and authorization before admitting reuse, and separates cache namespaces across incompatible sharing domains.

These properties concern admission and publication. We assess behavior with paired free-running outputs and task scores, and efficiency with native hits and complete service wall time.

IV. CONTRACTKV DESIGN

ContractKV implements a compile–certify–promote–reuse lifecycle over the engine's native prefix cache. The layout compiler creates the opportunity; the certifier establishes behavioral evidence; PSTP publishes an authorized prefix; and the service-template registry lets subsequent requests reuse the result. Qualification and physical cache residency have separate lifetimes.

A. An End-to-End Request Example

Figure 1 follows a retrieval template whose changing observation precedes a stable document. CALC moves eligible document content ahead of the observation. Under query-qualified execution, the first raw business response supplies a witness. A successful check commits the stable blocks; later authorized requests in that query scope select the same layout, reuse resident blocks, and compute their private suffixes. A new query requires a separate decision, but may map to the same stable-prefix identity. Rejection preserves the served raw response and promotes no candidate state.

B. Contract-Aware Layout Compiler

CALC operates on the registered dependency graph. Stable segments may be split into contiguous, character-preserving units to construct a bounded layout ladder. A stable-priority topological traversal moves only eligible units, respects dependency edges, and breaks ties by original order. For the 16-unit ladder, L1, L2, and L3 expose 4, 12, and 16 eligible units, respectively; L0 retains raw order. Full retokenization determines the actual stable-prefix boundary of each rendered candidate.

A source-disjoint development stage establishes the allowed model–family–layout combinations and freezes the policy before evaluation. The deployment studies use the registered ladder and its allowed descent policy. The final query-qualified implementation chooses one development-fixed candidate per scope and performs at most one GPU qualification attempt. Inherited L0 families retain raw execution.

The task-preserving renderer keeps essential fields and source whitespace intact (Section V). For an expanded graph $G' = (V', E')$, priority topological construction costs $O(|E'| + |V'| \log |V'|)$ per fixed candidate; model-backed qualification remains the dominant admission work.

C. Scoped Behavioral Certification

1) Witnesses and token-path checks: A behavioral witness is a completed model-generated output trajectory associated with a rendered input and execution identity. Query-qualified execution obtains it from the actual first raw business response, avoiding a separate raw generation solely to create a reference. Write its nonterminal witness tokens as $\mathbf{y} = (y_1, \ldots, y_L)$ and its natural terminal token as e; the recorded output IDs include both. The verifier evaluates the compiled prompt on the forced history $\mathbf{y}$, including termination.

For token z at output position i, define its next-token margin as

$$m_i(x, z) = \log p_\theta(z \mid x, \mathbf{y}_{<i}) - \max_{v \neq z} \log p_\theta(v \mid x, \mathbf{y}_{<i}). \quad (5)$$

The raw witness supplies its own per-position margins. The candidate must choose each witness token, and the terminal token, as its top choice. In the evaluated query-qualified protocol, the raw floor is 0.0625 and the candidate and terminal floors are 0.125:

$$\begin{aligned} m_i(x_{\mathrm{raw}}, y_i) &\geq \tau_{\mathrm{raw}}, \\ m_i(x_{\mathrm{cand}}, y_i) &\geq \tau_{\mathrm{cand}}, \quad i = 1, \ldots, L, \\ m_{L+1}(x_{\mathrm{cand}}, e) &\geq \tau_{\mathrm{cand}}. \end{aligned} \quad (6)$$

The raw terminal margin is also checked. Complete witness IDs, finite margins, the stop protocol, and the publication boundary are required for acceptance. The task-score threshold used in evaluation is separate from these model-level conditions.

For identical next-token distributions at verification and execution, agreement through the terminal token yields the same greedy sequence by induction. Finite-precision and execution-shape differences make this an execution condition, not a universal runtime theorem. Scope binding separately specifies which subsequent input instances may consume that evidence; free-running experiments measure their actual behavior. Greedy decoding, the output budget, stop settings, and model/runtime identity are part of the evaluated execution protocol.

2) Two evaluated qualification protocols: Deployment-qualified execution first qualifies a model–workflow–layout route on development trajectories. Registration then uses a representative *compiled-prompt* trajectory to attest and publish an immutable deployment object. Subsequent requests use that route under its registered template, identity, authorization, and lifecycle conditions. Behavioral support for later private inputs comes from the prospective qualification and the separately measured template-quality audit.

Query-qualified execution associates each decision with $(\textit{namespace}, q)$, where q hashes the exact query, system message, stable segments, model identity, generation budget and settings, renderer version, and policy. Its witness is the first real *raw-prompt* response in that scope. These protocols use the same compile/verify/publish architecture, but their witness origins and qualification granularity remain explicit.

The query-qualified experiments permit a declared metadata-only observation schema containing session identity, turn, and status, with no additional answer evidence. History and scratchpad segments are outside that schema. The shared-scope eligibility rule accepts a short, non-abstaining reference of at most 16 normalized words when it is a literal stable-context span, or when its raw minimum margin is strictly above 0.125. References failing this rule are classified as requiring an exact-observation scope; the evaluated trace declares no such repeated observations, so those requests remain raw. A further Python statement check is retained for the declared RepoBench subset. These conditions bound the evaluated empirical class; neither the metadata schema nor a positive margin proves invariance for every possible metadata value.

3) Compact in-engine verification: The verifier runs on the serving model backend. Previously approved, hash-matching resident prefix blocks can be reused during verification; a one-block guard and the answer/stop checks protect the verification boundary. The executor collects the rows needed for the witness and termination rather than exporting full prompt-logprob tensors. It returns a compact decision and publication record.

D. Prefix-Selective Transactional Promotion

PSTP makes a verifier decision a shared-cache event. Candidate KV initially resides outside the shared prefix index. The state machine advances through PRIVATE, VERIFYING, VERIFIED, and COMMITTED; unsuccessful paths end in ABORTED.

Algorithm 1 Single-shot query qualification. All work remains on the service clock.

Require: Registered Θ, request r, frozen policy
Render raw input; derive scope q; look up (Θ, q)
if an issued qualification matches this request **then**
  Check compiled-prefix identity; serve with native lookup
  return the business response
end if
Apply the pre-reference gate; serve r in raw order
if q is new and selected for qualification **then**
  Reserve q; retain the actual raw response as witness
  if witness/scope/output checks and post-reference gate pass **then**
    Verify one frozen candidate on witness tokens and EOS
    if PASS and valid publication binding **then**
      PSTP commits the eligible prefix; record qualification
    else
      Abort with zero candidate promotion; record rejection
    end if
  else
    Record the raw-route decision without GPU qualification
  end if
end if

Let L_s be the candidate's stable boundary, L_{cert} the authorized prompt-prefix boundary, and L_{lcp} the exact token LCP between the candidate request and its certified prompt. The promoted length is

$$L_{\mathrm{commit}} = B \left\lfloor \frac{\min(L_s, L_{\mathrm{cert}}, L_{\mathrm{lcp}})}{B} \right\rfloor. \tag{7}$$

The scheduler-thread insertion of the eligible block hashes is the publication linearization point. The corresponding KV tensors are already allocated; commit changes their visibility in the shared index rather than copying the tensors.

A non-pass, incomplete result, timeout, duplicate transaction, identity conflict, or invalid scope prevents new candidate publication. Cleanup releases private candidate resources. Raw execution can still populate the engine's ordinary cache, so the transactional property is about candidate promotion rather than freezing the entire cache domain.

After a successful commit, the native cache manager governs residency and reference counts. Its reclamation of an unreferenced block removes a physical reuse opportunity without invalidating the behavioral evidence. A later admitted request can recompute a missing prefix under the qualified layout. Section VI-F verifies this separation using actual lookup, publication, and reclamation events.

E. Service-Template Certificates and Amortization

A service-template certificate organizes the identity and lifecycle of reusable compiled state. The deployment representation is

$$C_\Theta = \langle I, V, H_s, L, P, Q, A, E, \Sigma \rangle, \tag{8}$$

I identifies the model/runtime execution, V the template version, H_s its stable content, L its layout, P the verifier policy, Q the qualification manifest, A the authorization domain, E the epoch, and Σ the authentication tag. The authenticated deployment path uses HMAC-SHA256 and checks the complete binding before forwarding a request.

The query-qualified implementation records joint registration decisions at $(\textit{namespace}, q)$ and stable-prefix associations at $(\textit{namespace}, H_s)$. This lets several qualified queries refer to the same reusable prefix and lets authorized tenants consume the same query decision. Changes to the bound identity or scope prevent reuse of the old decision; deployment lifecycle controls enforce revocation and re-registration for changed deployment objects.

The amortization units are explicit. For N sessions sharing one deployment-qualified template, session-scoped registration performs $O(N)$ attestations, whereas deployment registration performs $O(1)$ attestation plus per-request checks. For T template identities, U observed query scopes, and M requests, the query-qualified controller maintains $O(T+U)$ state and $O(U)$ scope decisions, with at most one GPU attempt per selected scope. For a fixed query directory, increasing tenant fan-out increases reuse without increasing the directory's qualification count.

F. Benefit Admission

Qualification consumes service resources before later requests realize savings. ContractKV therefore separates the decision to *invest in evidence* from the decision to *accept the candidate*. A frozen benefit policy estimates remaining authorized uses, marginal prefill savings, information-gathering cost, and GPU verification cost. In the two-stage query-qualified controller, the pre-reference estimate has the form

$$\widehat{G}_{\mathrm{pre}} = \widehat{p}_E \widehat{p}_{P|E} \widehat{R} \widehat{\Delta t} - \widehat{C}_{\mathrm{info}} - \widehat{p}_E \widehat{C}_{\mathrm{GPU}}. \tag{9}$$

After an eligible witness is available, the GPU decision uses the conditional pass estimate and the full predicted GPU cost. Observed prefix publication selects the calibrated cold/warm cost state, and the gate discounts native reuse already available to raw execution. Predictions are frozen before held-out evaluation.

The evaluated default applies a predicted waiting-budget test in cold sessions and uses expected aggregate benefit in template fan-out. We evaluate a separate length-aware waiting-budget variant in Section VI-G. Benefit admission can reduce qualification attempts; it never substitutes for the token-path or publication checks.

V. Implementation and Measurement

A. Integration with the Serving Engine

The prototype targets the vLLM 0.10.2 V1 scheduler and cache manager. It comprises a request-side compiler and qualification controller, an in-engine verification executor, and PSTP hooks at cache-index publication. Model weights and attention kernels remain unchanged. Candidate identities are derived after full prompt rendering and tokenization. Verification and compiled business requests use the policy hash and template namespace in their cache salt.

The implementation boundaries match the evaluated paths. The authenticated deployment path checks a signed template binding before forwarding requests and exercises version, authorization, and lifecycle validation. The query-qualified experiments use a controlled client with frozen model/runtime

identities, exact query-scope decisions, and stable-prefix checks, connected to the in-engine verifier and PSTP backend. These are two concrete integration paths: authenticated deployment admission and controlled query-qualified execution. Their shared verifier/PSTP backend and distinct request-side checks are evaluated separately.

Compiler and publication overheads measured in the deployment implementation are 0.389 ms mean for CALC, 0.249 ms for the residency lookup, 0.109 ms for the template-scope check, and 0.636 ms for PSTP finalization. The query-qualified studies measure their complete controller path afresh rather than importing these component timings as constants.

B. Task-Preserving Input Construction

The input adapter first attempts to retain each source in full within the common 16K window. It protects the complete question and code-completion prefix. If optional context exceeds the window, a predetermined head/tail rule selects original paragraph or code-line substrings, preserving order, whitespace, and an omission ledger. The selection does not inspect reference answers, model outputs, scores, margins, or qualification outcomes. Adjacent pieces from a single source segment are rendered contiguously in both raw and compiled prompts.

Both routes consume the same retained task content. Each source is checked against all declared model, tenant, turn, and selected-layout conditions, including the chat template and output reserve. Independent tokenization is then compared with actual API prompt IDs. The resulting audit covers the exact text and token sequence received by the serving engine. The common offline benchmark-packing cost is recorded separately; per-request rendering, planning, and verification remain in the online service measurement.

The earlier adapter used a fixed word budget and whitespace normalization. Our four-cell input ablation retains that adapter as a separate condition. This makes the effect of restoring task structure measurable without altering admission or scoring. Historical results using the earlier adapter are identified by protocol rather than pooled with the task-preserving study.

C. Execution Environment and Timing

The primary hardware is two RTX 3090 GPUs with 24 GB each. Single-model cells use one GPU; Qwen3-14B uses tensor parallelism across two GPUs in the deployment evaluation. The query-qualified environment is Python 3.10.20, PyTorch 2.8.0+cu128, vLLM 0.10.2, transformers 4.55.2, Triton 3.4.0, and FP16. Model checkpoints, tokenizers, runtime files, inputs, and policy identities are recorded per cell. The primary task-preserving cells use two concurrent requests; ten registered tenants per source describe sharing fan-out, not ten simultaneous GPU requests.

A fresh server process is used for each cell. The closed-loop client runs one tenant pair through ten rounds, then advances to the next pair. The next round becomes ready when the preceding business wave finishes; synchronous qualification performed between waves contributes to subsequent readiness delay. For request r,

$$T_{\mathrm{TTFT}}^{\mathrm{ready}}(r) = t_{\mathrm{first}}(r) - t_{\mathrm{ready}}(r). \tag{10}$$

Full service wall includes all business responses, first-use raw witnesses, planning, information and verification calls, unsuccessful attempts, waiting, scoring, logging, and final drain. Server startup, common pre-measurement infrastructure checks, configuration loading, and immutable input preparation have separately declared boundaries.

Closed-loop throughput is completed business requests divided by full service wall. The paired multiplier is the raw wall divided by the ContractKV wall for an equal request population. In the separate fixed-arrival deployment studies, SLO goodput counts responses meeting a specified readiness deadline per unit service time. Their arrival traces and workload protocol are reported separately.

Historical broad deployment measurements use an explicit residual-verification accounting rule,

$$C_{\mathrm{exposed}} = \max(C_{\mathrm{verify}} - \Delta_{\mathrm{slack}}, 0), \tag{11}$$

with 2,000 ms slack for Qwen/Mistral and 6,000 ms for the broad Llama protocol. All new query-qualified results use the directly measured complete service clock with zero slack. Component RPC sums provide a work breakdown alongside the directly measured wall clock.

D. Physical Accounting and Reproducibility

At initial scheduler allocation, the instrumentation joins each request's actual prompt IDs to its native hit count h_r and records $n_r - h_r$ as newly required prefill tokens. Verification requests have separate counters. These are allocation-level workload measures; latency is measured directly rather than inferred from token counts.

Source groups, policy settings, and selection rules are frozen before each independent cohort is opened. Repetitions use new processes and recorded device/policy assignments. Statistical summaries preserve source-template clusters and process repetition; tenant and query repetitions remain within their source cluster. The artifact supplies request-level data, model/runtime manifests, source snapshots, scripts, and archive checksums. The supplemental record preserves the extended tables and unsuccessful variants.[1]

VI. Evaluation

The evaluation follows the service object's lifecycle. We first test whether qualification repays its full cost on preserved tasks, then examine behavior and coverage, isolate publication governance and amortization, and explain the effects of load and residency. Query-qualified results form the primary evidence; deployment-qualified results retain their separate protocol and accounting.

[1]https://github.com/zhouz4884-coder/contractkv. The experimental evidence is pinned to commit `9bf891af`; full paths and hashes are listed in the supplementary evidence register.

A. Workloads, Controls, and Units

The primary cohort contains three HotpotQA and three RepoBench sources, chosen by frozen hash order after excluding 567 previously used sources [8], [9]. Reading-comprehension data come from the LongBench collections [10]. Each source defines a ten-tenant, ten-round template with three recurring query wrappers and changing private metadata. This is an explicitly repeated authorized directory, not a measured production query distribution.

We run Qwen3-8B, Mistral-7B-Instruct-v0.3, and Llama-3.1-8B-Instruct [11]–[13]. Both policies have the same 16K window, retained task content, greedy decoding, fixed seed, request order, and two-request wave clock. HotpotQA has a 128-token generation budget and RepoBench 384. *Raw* retains the original order with native APC; *ContractKV* uses the frozen candidate, eligibility, margin/EOS checks, and publication path. Mistral and Llama inherit the development-frozen RepoBench L0 route.

Every model/policy uses three fresh processes. Each run completes 600 business requests, including 540 future requests. Thus the study contains 10,800 business generations and 4,860 future raw/strategy pairs. Six sources are the independent content units; tenants, rounds, and processes are nested observations. Throughput ranges are process min–max; confidence intervals resample source-template clusters and repetitions. Separate cohorts and comparisons sharing raw controls are not pooled as independent trials.

We retain prompt/output IDs, task scores, routes, qualifications, timing, and cache counters. QA uses the frozen token-F1 scorer; code uses the existing SequenceMatcher-based edit-similarity adapter. These are declared benchmark adaptations. Material score changes use $|\Delta s| > 0.03$; exact ID agreement is measured separately. The supplement gives per-source utility, process values, clock definitions, and extended tables.

B. Does Strict Qualification Repay Its Full Cost?

We expect a net gain when enough admitted requests reuse a stable prefix to repay first-use qualification. Figure 2 tests this on the same source set, with no tool slack and successful and failed attempts included. Throughput improves by 1.619× on Qwen, 1.109× on Mistral, and 1.070× on Llama. The process ranges are 1.615–1.623×, 1.107–1.110×, and 1.068–1.072×. These are complete service gains rather than prefill-only estimates.

Figure 2(b) measures the work saved: including qualification, total new-prefill counts fall by 45.44%, 14.64%, and 10.89%. These scheduler observations are workload counts, not inferred GPU time. Directly measured mean readiness falls by 43.87%, 12.09%, and 3.99%.

The throughput result does not imply lower tails. All p99 point estimates increase (Table II); Mistral's source-cluster/repetition interval for p99 reduction is wholly negative, [-60.576, -0.137]%. Mean reduction intervals are [5.640, 79.479]%, [-0.350, 34.781]%, and [1.550, 5.724]%. The cohort therefore supports a full-cost execution benefit with source-dependent latency effects.

TABLE II
TAIL COST AND ADMITTED COVERAGE IN THE PRIMARY COHORT. READINESS STATISTICS CONCERN FUTURE REQUESTS. FULL PROCESS RANGES AND SOURCE-CLUSTER INTERVALS ARE IN THE SUPPLEMENT.

Model	Compiled / future pairs	p99 raw → CKV (ms)
Qwen	684 / 1,620	8,561.8 → 9,146.6
Mistral	342 / 1,620	9,418.2 → 9,511.3
Llama	270 / 1,620	7,851.3 → 7,905.7

A separate six-source Qwen cohort supplies an independent content check: unchanged ContractKV reaches 1.803× throughput (process range 1.798–1.808×) and 774/1,620 compiled future pairs. It is a second cohort, not an additional model bar or another repetition of Fig. 2.

C. Does Qualification Preserve Observed Behavior?

Compilation must obtain its gain within the admitted behavioral scope. Figure 3 reports both scope coverage and free-running evidence. All 1,296 compiled future pairs have identical output IDs and unchanged scores. Including initial business rounds gives 1,440 compiled pairs with the same outcome. HotpotQA contributes compiled requests for every model; Qwen additionally admits RepoBench, whereas Mistral/Llama keep the inherited L0 route.

Behavioral preservation and absolute utility remain different outcomes. Qwen's 684 admitted raw references all receive positive score credit; the counts are 258/342 for Mistral and 90/270 for Llama. Llama preserves 180 zero-F1 future references from two HotpotQA sources, and Mistral preserves 84 from one source. Neither exact agreement nor positive partial credit establishes semantic correctness.

Across all business routes, material score increases/decreases are 0/0 for Qwen, 2/1 for Mistral, and 5/3 for Llama. All changes occur on L0; the Mistral decrease is in an initial request and the Llama decreases are future requests. Corresponding output variants appear in scheduled raw-only repeats, demonstrating background variation, but not proving that every L0 event is unrelated to policy-associated execution conditions.

The historical audit in Fig. 3(b) retains 2,442 compiled future pairs from a 50-source, five-family protocol: 2,178 RULER, 174 NarrativeQA, and 90 RepoBench [14]–[16]. It has zero compiled ID changes and 33 full-route material decreases on L0. Its older input adapter and source mix differ from the main cohort. RULER supports that separate audit; it does not substitute for the common non-RULER evidence in panel (a).

a) Why the scope matters.: Earlier broad qualification admitted later wrappers and produced 108 material decreases among 3,672 compiled pairs. A later Qwen NarrativeQA case changed its raw answer under an allowed turn-field change while the compiled answer retained the witness, despite a 0.40625 candidate margin. The final eligibility class was frozen after this development diagnosis and before the primary sources were opened. A margin alone cannot justify extending evidence to unseen dynamic inputs.

b) Actual output horizon.: Separate conditions produce and verify naturally terminated outputs of 642 tokens for Qwen, 769 for Mistral, and 578 for Llama, each with six correct

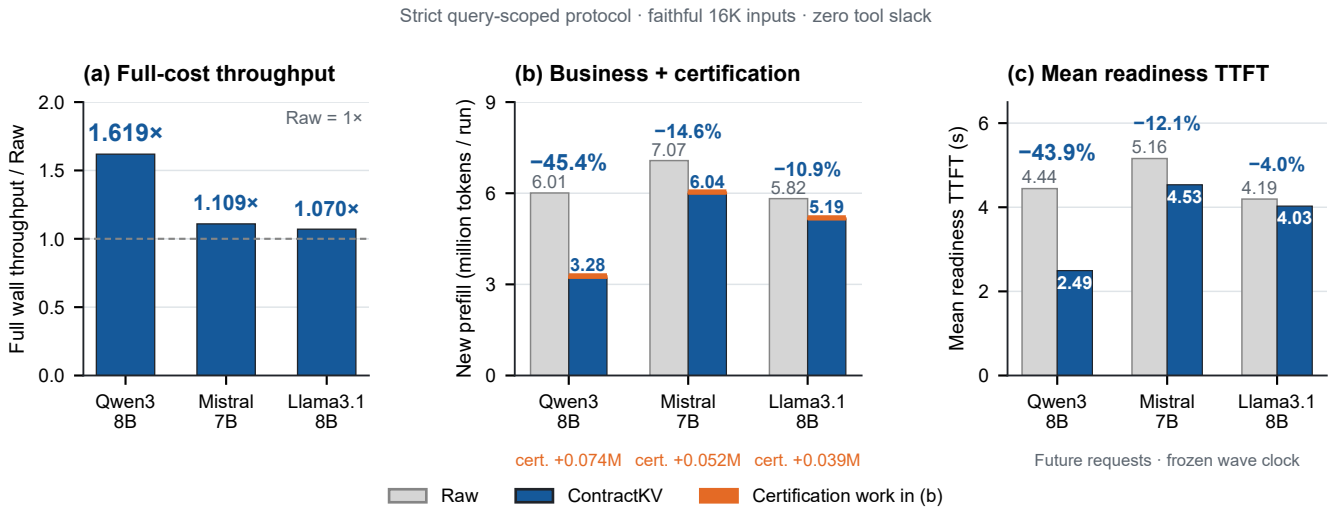


Fig. 2. Strict query-qualified performance on six source-disjoint HotpotQA/RepoBench templates, three processes/model, faithful 16K inputs, and zero slack. (a) Throughput charges all business responses, qualification, failed attempts, and synchronous waiting. (b) Actual business plus qualification new-prefill counts per run; orange caps mark certification work at its true scale, with exact added counts below. (c) Mean readiness TTFT over future requests uses the frozen two-request wave clock. Three-repeat values and tail statistics are reported separately.

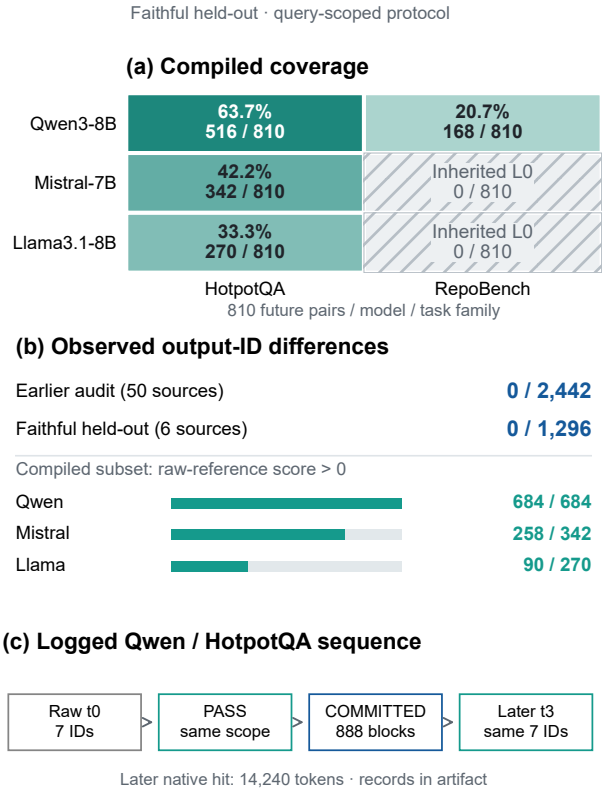


Fig. 3. Qualification coverage and free-running evidence. (a) Each model/task cell has 810 future pairs; hatched cells inherit L0. (b) Output-ID differences and raw-reference positive-score counts within compiled subsets. Positive credit is not complete correctness. (c) One logged Qwen/HotpotQA witness, PASS, committed prefix and later identical output; full records are archived.

future pairs across three processes. They use explicitly declared 128/128/192-item schemas. Llama's 224-item condition omits the final 100 items and remains a task failure. These results establish actual 512+ output conditions, not universal long-output correctness.

D. What Changes When Task Inputs Are Preserved?

The primary protocol follows a controlled Qwen input ablation on two HotpotQA and two RepoBench development sources. Old-adapter and preserved-input conditions each compare raw and ContractKV under the same 16K window, two-request concurrency, admission, scorer, and clock. Restoring task structure raises raw HotpotQA F1 from 0 to 0.22222 and code score from 0.12067 to 0.36667. Compiled coverage rises from 30/360 to 112/360 and full-wall throughput from 1.039× to 1.117×, with unchanged compiled IDs.

This diagnoses an input effect, not an accuracy improvement caused by ContractKV. The older adapter removes answer literals present in stored context for eight of ten audited HotpotQA sources, empties five of ten code prefixes, and flattens code whitespace in all ten. Literal availability is not answerability, but losing the completion prefix changes the task. The revised renderer protects task fields and original whitespace, with answer-blind cropping of optional context. The independent primary cohort validates the whole preserved-input pipeline; historical results are retained under their original protocol.

E. What Do Publication Control and Amortization Add?

A frozen safe layout isolates available reuse, whereas governance should determine which candidates acquire shared visibility. Figure 4 examines both questions on deployment-qualified controls. D1 uses a prequalified layout without online registration; D2 adds verification and transactional registration. Mean 500-ms SLO goodput is 1.472 versus 1.428 requests/s, retaining 97.0% of D1's mean. Their TTFTs are 194.95 and 194.72 ms. D2 is not claimed to outperform an already safe D1 route.

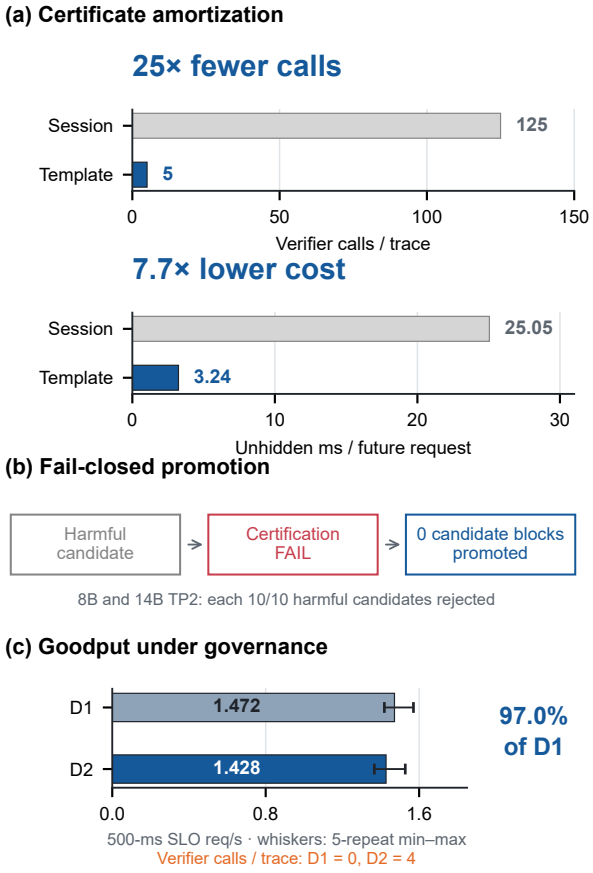


Fig. 4. Separate deployment controls. (a) Registration granularity with fixed layout/locality; three processes and 1,125 future requests/trace. (b) Each 8B/14B TP2 probe rejects 10/10 harmful candidates with zero rejected-candidate promotion. (c) Same-engine D1/D2 500-ms SLO goodput; whiskers are five-process min–max, not confidence intervals.

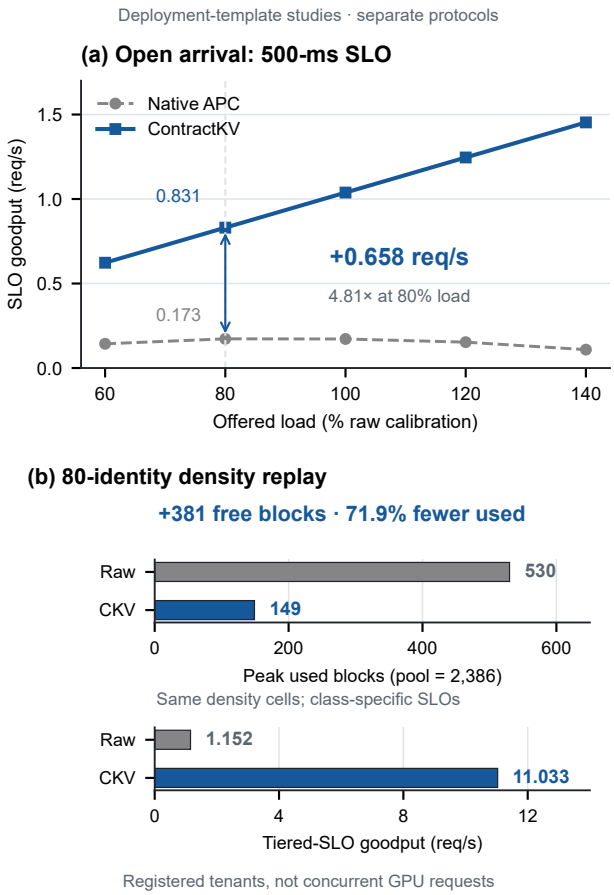


Fig. 5. Distinct deployment protocols. (a) Open-arrival 500-ms SLO goodput; the 4.81× ratio is the 80%-load point. (b) The same 80-identity density cells supply both allocator occupancy and goodput under class-specific 750/1,500/3,000-ms SLOs, with a 2,386-block pool. Registered identities are not simultaneous GPU requests.

Behaviorally harmful, dependency-valid candidates provide the complementary test. In each of two controlled 20-case sets, all ten harmful candidates are rejected with zero newly promoted candidate tokens. A 14B control also checks actual post-decision outputs against raw scores. Live source-hook, concurrency, cancellation, duplicate, timeout, and authenticated-binding controls exercise the publication boundary. The separate 325,000-operation stress study tests the in-memory state machine and is not counted as GPU serving evidence.

1) The unit of amortization: Qualification becomes economically useful when its object boundary matches authorized reuse. At 25-tenant locality and the frozen 80%-load deployment point, template registration replaces 125 session registrations with five template registrations. Fan-out rises from 9 to 225 requests/object; unhidden verification exposure falls 25.05 to 3.24 ms/request. Prefix hits stay close at 97.82%/98.19%, indicating that the main change is admission granularity rather than layout.

Query-qualified execution has a different amortization unit. The second Qwen cohort obtains 774 compiled future pairs from 27 issued scope-repeat decisions and 42 total GPU calls, including failures. Its cost grows with distinct selected scopes, then amortizes across tenants using them. This preserves the template prefix as the physical object without assigning deployment's $O(1)$ behavioral cost to arbitrary dynamic queries.

F. Where Do Service and Residency Benefits Hold?

Figure 5 tests a separately qualified deployment workload at fixed arrival loads and in bounded-concurrency density replay. It shows absolute SLO goodput as well as allocator occupancy, rather than presenting closed-loop throughput as capacity. At 80 identities, used blocks decrease by 71.9%, leaving 381 more free blocks. Density goodput and block values come from the same cells, but do not isolate headroom as the sole cause of improved throughput. These marker-bearing service traces have a separate task-quality audit and are distinct from the task-bearing primary cohort.

Figure 6 explains why sharing count alone is insufficient. Benefit rises sharply over an observed sharing interval, but round-robin access to a larger working set nearly eliminates it. A qualified object still needs resident physical blocks. In the primary cohort, 189 compiled business requests have incomplete hits: 135 Llama zero hits, 42 Mistral zero hits, and 12 Qwen partial hits. All 1,440 compiled business requests match the certified prefix IDs and commit boundary; each incomplete-hit request has a later full-hit peer in its two-request wave.

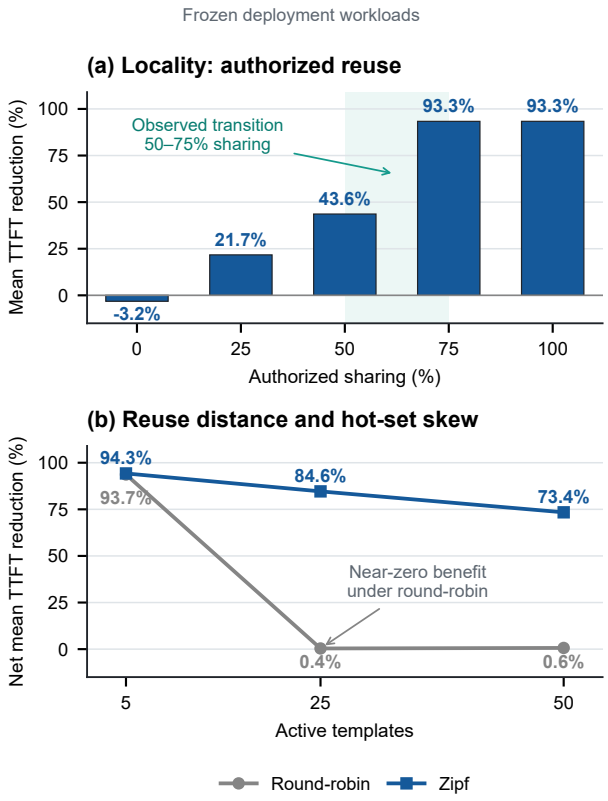


Fig. 6. Operating region in frozen deployment workloads. (a) The shaded 50–75% sharing interval is an observed transition, not a universal threshold. (b) Access-pattern sensitivity to reuse distance and hot-set skew. Both axes report TTFT reduction; neither is a direct residency fraction.

To test the physical cause, three frozen known-source probes replay archived IDs with read-only instrumentation of lookup, publication, and reclamation. Across 138 business requests and four successful qualifications, all 13 incomplete hits follow deletion of the last matching native mapping at reference count zero. Salt and hash chains match the certificate, while the mapping is absent before lookup. All 12 immediate post-commit reuse controls fully hit. These events match the native cache reclamation design [17]. They diagnose the three sources, not every historical event, and do not supply a new performance number. Figure 2 already charges misses and recomputation.

G. Cost Policies and Cross-Stack Comparisons

Long-input qualification can stall future service. A development example predicts 2.027 s conditional GPU cost but measures a 4.034 s RPC and 4.142 s controller; the lower eligibility-weighted pre-reference prediction is a different quantity. A frozen length-bucket/waiting-budget variant reduces calls from nine to six without losing the development cohort's 112 compiled pairs, lowering RepoBench p99 from 10.135 to 6.171 s.

On the independent second Qwen cohort, however, its profiling support rejects useful long inputs. Coverage falls from 774 to 258 compiled pairs, and throughput from 1.803× to 1.070×. Global p99 falls from 13,386.9 to 8,801.9 ms, but the direct source/repetition interval for reduction, [-12.145, 37.569]%, crosses zero. All eight/six material decreases remain on L0; five strategy/position variants are absent from three raw controls and remain unexplained. We retain the original gate as the main measured configuration. The result identifies a cost/coverage tradeoff rather than a generally successful tail fix.

Historical deployment results report 29.41–44.69% net mean TTFT reduction with the explicit 2s/6s residual-verification slack rule. A separate zero-slack supplement reports 17.50%, −2.84%, 7.69%, and 10.59% for Qwen8B, Mistral7B, Llama8B, and Qwen14B TP2. These operating regimes are kept separate from the zero-slack primary study.

1) Matched-native quality: Cross-position methods require their own matched-native references. A 9,000-pair quality study has 2,250 pairs/method: CacheBlend records 224 material decreases and 331 increases, with mean score 0.333210→0.339817; ContractKV records two decreases and no increases, with 0.353780→0.353744. Different raw means, runtimes, prompt construction, and separators preclude a direct speed ranking or multiplicative causal risk claim. The supplement retains both directions and threshold sensitivity. Mean task utility and per-request behavioral preservation answer different questions; same-engine D1/D2 supplies the performance attribution.

VII. Related Work

Physical and cross-position reuse. PagedAttention provides block-based KV memory management and sharing [2]; SGLang's RadixAttention organizes reusable prefixes for structured language-model programs [3]. Prompt Cache exposes reusable modules through explicit schemas [4]. CacheBlend selectively recomputes information when combining cached passages [5], while EPIC uses position-independent composition with LegoLink [6]. CacheSlide and MiniPIC further explore cross-position or position-independent reuse [18], [19]. These systems establish a rich space of reuse mechanisms. ContractKV contributes a control-plane protocol that couples a dependency-valid layout, behavioral evidence, and an authorization-bound commit before transformed state enters that space as a shared object.

Application and service scheduling. Parrot's Semantic Variables expose application dataflow to the serving system [7]. Llumnix dynamically schedules serving requests across instances [20]; Sarathi-Serve addresses prefill/decode throughput–latency interference [21]; and Preble combines prompt reuse with distributed load balancing [22]. DistServe and Mooncake separate serving stages or distribute KV storage and computation [23], [24]. These mechanisms govern resource placement and execution. ContractKV instead determines when a compiled representation is eligible for publication and authorized reuse within one cache domain. Its waiting and residency measurements identify where model-backed qualification and native scheduling interact.

Cost models, contracts, and cache privacy. Vidur combines experimental profiling with predictive models to estimate serving behavior [25]; our cost-policy study makes the support-domain and coverage consequences of a small frozen profile

explicit. Classical service contracts such as WSLA associate measurable service obligations with monitoring and enforcement [26]. ContractKV binds a reusable model-execution artifact to evidence, identity, authorization, and a transactional boundary. Work on prompt leakage and audits of prompt-cache sharing demonstrates that cache timing is itself an observable privacy channel [27], [28]. Those results motivate separating authorization-controlled cache state from protection against timing inference.

VIII. Discussion and Limitations

A. What the Service Contract Establishes

The reusable object couples layout, evidence, identity, and publication; its benefit depends on matching requests and resident blocks. Token-path certification supplies relative behavioral evidence under the declared greedy execution protocol. It is not an oracle for answer correctness or invariance over arbitrary future inputs. A zero-F1 witness can be preserved exactly, which is why Fig. 3 reports reference utility separately from output-ID agreement.

Query-qualified execution assumes a repeated authorized directory and typed metadata-only observations. It does not cover arbitrary evolving histories or unrestricted new tool evidence. If the number of unique query scopes grows with the request count, qualification has little opportunity to amortize. Production query-repeat frequencies were not measured. Deployment-qualified registration has a different witness origin and reuse unit; the implementation does not automatically switch between two independent endpoint certificate systems.

B. Trust, Authorization, and Timing

The operator manifest is authoritative for segment stability and sharing permission. A hash establishes content identity, not whether content was correctly declared public. In nine CPU registration cases, seven invalid declarations are rejected; a legitimate declaration and a trusted operator's semantically incorrect declaration are accepted. This helper remains a separate validation test, rather than an integrated live endpoint. The authenticated deployment path and in-engine PSTP controls are evaluated at their stated boundaries.

Authorization-bound namespaces do not remove timing inference within an allowed sharing domain [27], [28]. ContractKV does not implement a timing-channel defense. P4 therefore describes cache-state authorization, not a comprehensive application confidentiality boundary.

C. Workload and Platform Boundaries

The main study uses six independent content sources, two-request concurrency, greedy decoding, and 24-GB GPUs. Tenants and process repeats do not replace new source templates. The preserved-input adapter uses answer-blind optional-context cropping and the declared legacy code scorer. Its Python-output check excludes some legal next-line fragments. The artifact retains omitted spans, inherited L0 routes, and all score changes.

The native-cache probes diagnose three known sources with synchronous instrumentation; their timings are not performance measurements, and their traces cannot retrospectively attribute every historical miss. The separate query-qualified fixed-arrival extension completed five of 54 planned cells and is not used as a capacity result. A vLLM 0.25.1 audit checks relevant integration objects and layout opportunities, not a completed port. Distributed cache domains would require additional lifecycle and publication coordination.

The resulting operating lesson is specific: invest in behavioral qualification where authorized reuse can repay it, measure the cost on the service clock, and account for native reclamation when interpreting a qualified prefix. The original gate remains the principal measured configuration because the tested cost adaptation traded away useful coverage. These conditions define where the reported benefits hold.

IX. Conclusion

ContractKV turns a compiled KV prefix into an evidence-bound service object. CALC exposes dependency-valid stable prefixes, scoped certification determines admissible use, PSTP controls atomic visibility, and template identities let authorized requests share qualification and physical state. Source-disjoint, task-preserving experiments show $1.619\times$, $1.109\times$, and $1.070\times$ full-cost throughput with unchanged output IDs in all 1,296 admitted compiled future pairs. Publication controls and lifecycle traces explain how those gains arise and when they diminish. The service-level result is a measured route from repeated prompt content to qualified, transactionally controlled reuse.

References

[1] Azure Samples, "Azure AI Search + Azure OpenAI demo: Conversation builder and answer template," source revision `558ffd397edd`, *azure-search-openai-demo*. [Online]. Available: https://github.com/Azure-Samples/azure-search-openai-demo. Accessed: Oct. 17, 2026.

[2] W. Kwon *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. ACM SIGOPS Symp. Oper. Syst. Principles (SOSP)*, 2023, pp. 611–626, doi: 10.1145/3600006.3613165.

[3] L. Zheng *et al.*, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[4] I. Gim, G. Chen, S.-s. Lee, N. Sarda, A. Khandelwal, and L. Zhong, "Prompt Cache: Modular attention reuse for low-latency inference," in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 6, 2024.

[5] J. Yao *et al.*, "CacheBlend: Fast large language model serving for RAG with cached knowledge fusion," in *Proc. 20th Eur. Conf. Comput. Syst. (EuroSys)*, 2025, doi: 10.1145/3689031.3696098.

[6] J. Hu *et al.*, "EPIC: Efficient position-independent caching for serving large language models," in *Proc. 42nd Int. Conf. Mach. Learn. (ICML)*, PMLR, vol. 267, 2025, pp. 24391–24402.

[7] C. Lin, Z. Han, C. Zhang, Y. Yang, F. Yang, C. Chen, and L. Qiu, "Parrot: Efficient serving of LLM-based applications with semantic variable," in *Proc. 18th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, 2024, pp. 929–945.

[8] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning, "HotpotQA: A dataset for diverse, explainable multi-hop question answering," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2018, pp. 2369–2380, doi: 10.18653/v1/D18-1259.

[9] T. Liu, C. Xu, and J. McAuley, "RepoBench: Benchmarking repository-level code auto-completion systems," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[10] Y. Bai *et al.*, "LongBench: A bilingual, multitask benchmark for long context understanding," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2024, pp. 3119–3137, doi: 10.18653/v1/2024.acl-long.172.

[11] A. Yang *et al.*, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.
[12] A. Q. Jiang *et al.*, "Mistral 7B," *arXiv preprint arXiv:2310.06825*, 2023.
[13] A. Dubey *et al.*, "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.
[14] C.-P. Hsieh *et al.*, "RULER: What's the real context size of your long-context language models?" in *Proc. Conf. Language Modeling (COLM)*, 2024.
[15] T. Kočiský *et al.*, "The NarrativeQA reading comprehension challenge," *Trans. Assoc. Comput. Linguistics*, vol. 6, pp. 317–328, 2018, doi: 10.1162/tacl_a_00023.
[16] P. Dasigi, K. Lo, I. Beltagy, A. Cohan, N. A. Smith, and M. Gardner, "A dataset of information-seeking questions and answers anchored in research papers," in *Proc. Conf. North American Chapter Assoc. Comput. Linguistics: Human Language Technologies (NAACL-HLT)*, 2021, pp. 4599–4610, doi: 10.18653/v1/2021.naacl-main.365.
[17] vLLM Project, "Automatic prefix caching," *vLLM Documentation*, ver. 0.10.2. [Online]. Available: https://docs.vllm.ai/en/v0.10.2/design/prefix_caching.html. Accessed: Oct. 17, 2026.
[18] Y. Liu *et al.*, "CacheSlide: Unlocking cross position-aware KV cache reuse for accelerating LLM serving," in *Proc. 24th USENIX Conf. File Storage Technol. (FAST)*, 2026, pp. 83–99.
[19] N. Ordonez and T. Parnell, "MiniPIC: Flexible position-independent caching in $<$100 LOC," *arXiv preprint arXiv:2606.13126*, 2026.
[20] B. Sun *et al.*, "Llumnix: Dynamic scheduling for large language model serving," in *Proc. 18th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, 2024, pp. 173–191.
[21] A. Agrawal *et al.*, "Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve," in *Proc. 18th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, 2024, pp. 117–134.
[22] V. Srivatsa, Z. He, R. Abhyankar, D. Li, and Y. Zhang, "Preble: Efficient distributed prompt scheduling for LLM serving," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.
[23] Y. Zhong *et al.*, "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *Proc. 18th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, 2024, pp. 193–210.
[24] R. Qin *et al.*, "Mooncake: Trading more storage for less computation—A KVCache-centric architecture for serving LLM chatbot," in *Proc. 23rd USENIX Conf. File Storage Technol. (FAST)*, 2025, pp. 155–170.
[25] A. Agrawal *et al.*, "Vidur: A large-scale simulation framework for LLM inference," in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 6, 2024.
[26] A. Keller and H. Ludwig, "The WSLA framework: Specifying and monitoring service level agreements for Web services," *J. Network Syst. Manage.*, vol. 11, no. 1, pp. 57–81, 2003, doi: 10.1023/A:1022445108617.
[27] G. Wu, Z. Zhang, Y. Zhang, W. Wang, J. Niu, Y. Wu, and Y. Zhang, "I know what you asked: Prompt leakage via KV-cache sharing in multi-tenant LLM serving," in *Proc. Network Distrib. Syst. Security Symp. (NDSS)*, 2025, doi: 10.14722/ndss.2025.241772.
[28] C. Gu, X. L. Li, R. Kuditipudi, P. Liang, and T. Hashimoto, "Auditing prompt caching in language model APIs," in *Proc. 42nd Int. Conf. Mach. Learn. (ICML)*, 2025. Author version: arXiv:2502.07776.